Белорусский государственный университет

Физический факультет

Лабораторная работа 1

Численные методы решения СЛАУ

Выполнила: Роговая М.К.

Преподаватель: Тимощенко И.А.

Дата: 27.10.2026

Минск, 2026

Введение

Структура повторяет задание: **разминка** (метод прогонки), **основное блюдо** (Якоби, Зейдель, верхняя релаксация на матрице из PyAMG), **десерт** (исследование параметра ω).

Как запускать. Нужны `numpy`, `scipy`, `matplotlib` и `pyamg`. Ячейки выполняются сверху вниз (*Restart Kernel and Run All Cells*). Размер сетки, точность и лимит итераций собраны в первой ячейке с настройками.

In [1]:
```python
# Требуется: numpy, scipy, matplotlib, pyamg
import time
import numpy as np
import scipy.sparse as sp
import scipy.sparse.linalg as spla
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)          # seed для повторяемости
plt.rcParams.update({"figure.figsize": (10, 6), "axes.grid": True})

# ---------- настройки ----------
GRID = 100          # сетка для PyAMG (по заданию не меньше 100 на 100)
NU = 0.34           # коэффициент Пуассона (латунь)
E_MOD = 110e9       # модуль Юнга (латунь)
TOL = 1e-6          # критерий остановки: ||b - Ax|| / ||b|| < TOL
MAX_ITER = 20000    # аварийный лимит итераций
ALPHA = 1e-3        # малый сдвиг диагонали, если матрица вырождена (см. ниже)
```

1. Разминка: метод прогонки

Для трёхдиагональной системы $l_i x_{i-1} + d_i x_i + u_i x_{i+1} = f_i$ ищем $x_i = \alpha_i x_{i+1} + \beta_i$.

Прямой ход: $\alpha_0 = -u_0/d_0,\ \beta_0 = f_0/d_0$, далее $\alpha_i = \dfrac{-u_i}{d_i + l_i\alpha_{i-1}}, \quad \beta_i = \dfrac{f_i - l_i\beta_{i-1}}{d_i + l_i\alpha_{i-1}}$.

Обратный ход: $x_{N-1} = \beta_{N-1}$, $x_i = \alpha_i x_{i+1} + \beta_i$.

Матрицу храним тремя векторами (а не плотным массивом $N \times N$), иначе память и время растут как N^2 и асимптотику $O(N)$ проверить нельзя. Точное решение $\bar{x} = [0, 1, \ldots, n]$, то есть $N = n + 1$ компонент.

In [2]:
```python
def tridiagonal_random(N, rng, lo=-50, hi=50):
    """Случайная трёхдиагональная матрица со СТРОГИМ диагональным преобладанием.
    lower[i] = A[i, i-1] (lower[0] = 0), upper[i] = A[i, i+1] (upper[-1] = 0)."""
    lower = rng.integers(lo, hi + 1, N).astype(float)
    upper = rng.integers(lo, hi + 1, N).astype(float)
    lower[0] = 0.0
    upper[-1] = 0.0
    margin = rng.integers(1, 51, N)       # запас > 0 => строгое преобладание
```

```
    sign = rng.choice([-1.0, 1.0], N)
    diag = sign * (np.abs(lower) + np.abs(upper) + margin)
    return lower, diag, upper


def tridiagonal_to_sparse(lower, diag, upper):
    return sp.diags([lower[1:], diag, upper[:-1]], [-1, 0, 1], format="csr")


def is_strictly_dominant(lower, diag, upper):
    return bool(np.all(np.abs(diag) > np.abs(lower) + np.abs(upper)))


def thomas(lower, diag, upper, f):
    """Метод прогонки, O(N). Списки Python быстрее поэлементного доступа к numpy."""
    n = len(diag)
    l, d, u, f = lower.tolist(), diag.tolist(), upper.tolist(), f.tolist()
    alpha = [0.0] * n
    beta = [0.0] * n
    alpha[0] = -u[0] / d[0]
    beta[0] = f[0] / d[0]
    for i in range(1, n):                    # прямой ход
        denom = d[i] + l[i] * alpha[i - 1]
        alpha[i] = -u[i] / denom
        beta[i] = (f[i] - l[i] * beta[i - 1]) / denom
    x = [0.0] * n
    x[-1] = beta[-1]
    for i in range(n - 2, -1, -1):           # обратный ход
        x[i] = alpha[i] * x[i + 1] + beta[i]
    return np.array(x)
```

1.1 Проверка правильности

Мера сравнения: относительная погрешность в max-норме $\|x - \bar{x}\|_\infty / \|\bar{x}\|_\infty$ и относительная невязка $\|Ax - b\|_\infty / \|b\|_\infty$. Для сравнения решаем ту же систему `scipy.sparse.linalg.spsolve`.

In [3]:
```
n = 50
N = n + 1
lower, diag, upper = tridiagonal_random(N, rng)
A_tri = tridiagonal_to_sparse(lower, diag, upper)
x_exact = np.arange(N, dtype=float)             # [0, 1, ..., n]
b_tri = A_tri @ x_exact

x_thomas = thomas(lower, diag, upper, b_tri)
x_scipy = spla.spsolve(A_tri.tocsc(), b_tri)

def rel_err(u, v):
    return np.max(np.abs(u - v)) / np.max(np.abs(v))


print("строгое диагональное преобладание:",
      is_strictly_dominant(lower, diag, upper))
print("погрешность прогонки      :", rel_err(x_thomas, x_exact))
```

```
print("погрешность spsolve       :", rel_err(x_scipy, x_exact))
print("относительная невязка     :", rel_err(A_tri @ x_thomas, b_tri))
```

```
строгое диагональное преобладание: True
погрешность прогонки      : 1.4210854715202004e-16
погрешность spsolve       : 3.552713678800501e-16
относительная невязка     : 1.5115418011848566e-16
```

1.2 Время решения от размера системы

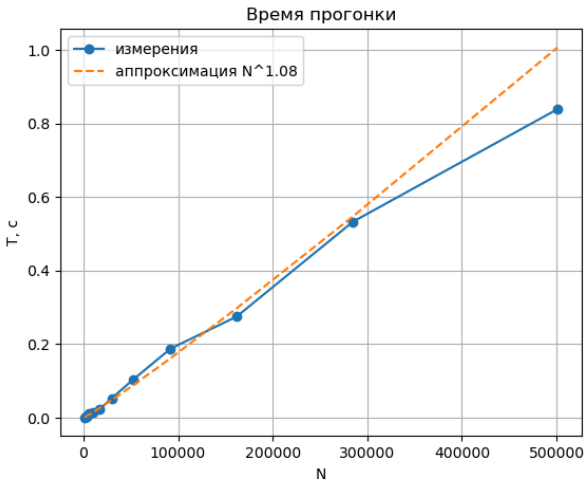
Время измеряем только для самой прогонки (генерацию матрицы в замер не включаем), берём минимум из нескольких запусков, чтобы убрать шум. Если метод $O(N)$, то на графике в двойном логарифмическом масштабе наклон должен быть близок к 1.

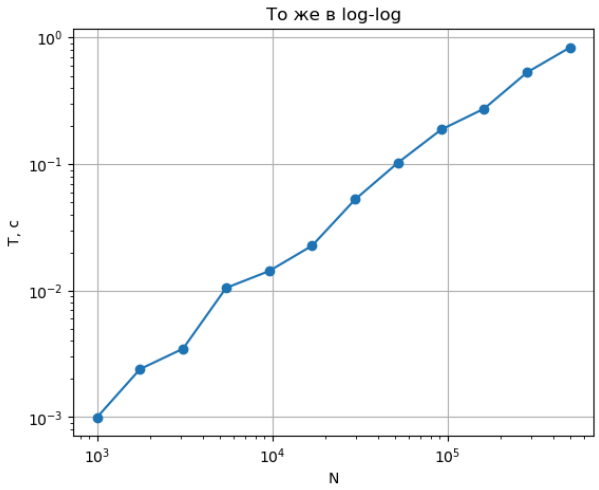
In [4]:
```
sizes = np.unique(np.logspace(3, 5.7, 12).astype(int))
times = []
errors = []
for N in sizes:
    lo_, di_, up_ = tridiagonal_random(N, rng)
    xe = np.arange(N, dtype=float)
    f = tridiagonal_to_sparse(lo_, di_, up_) @ xe
    best = np.inf
    for _ in range(3):
        t0 = time.perf_counter()
        xs = thomas(lo_, di_, up_, f)
        best = min(best, time.perf_counter() - t0)
    times.append(best)
    errors.append(np.max(np.abs(xs - xe)) / np.max(np.abs(xe)))
times = np.array(times)

slope, intercept = np.polyfit(np.log(sizes), np.log(times), 1)
print(f"наклон в log-log: {slope:.3f}  (для O(N) ожидается около 1)")
print("максимальная относительная погрешность по всем N:", max(errors))

fig, ax = plt.subplots(1, 2, figsize=(14, 5))
ax[0].plot(sizes, times, "o-", label="измерения")
ax[0].plot(sizes, np.exp(intercept) * sizes ** slope, "--",
           label=f"аппроксимация N^{slope:.2f}")
ax[0].set_xlabel("N"); ax[0].set_ylabel("T, c")
ax[0].set_title("Время прогонки"); ax[0].legend()
ax[1].loglog(sizes, times, "o-")
ax[1].set_xlabel("N"); ax[1].set_ylabel("T, c"); ax[1].set_title("То же в log-log")
plt.show()
```

```
наклон в log-log: 1.079  (для O(N) ожидается около 1)
максимальная относительная погрешность по всем N: 1.0452585431779952e-15
```





2. Основное блюдо: итерационные методы

Все три стационарных метода записаны в канонической форме $x^{(k+1)} = x^{(k)} + M^{-1}(b - Ax^{(k)})$, где $A = L + D + U$:

метод	M
Якоби	D
Зейдель	$D + L$
верхняя релаксация (SOR)	$D/\omega + L$

Это в точности формулы из задания. Матрицы хранятся разреженными, а M^{-1} для Зейделя и SOR применяется через один раз посчитанное LU-разложение треугольной матрицы (без перестановок, заполнения нет). Критерий остановки: $\|b - Ax^{(k)}\|_2 / \|b\|_2 < \mathrm{TOL}$, а не $\|x^{(k+1)} - x^{(k)}\|$, потому что именно невязка нужна для графика. Если невязка растёт выше 10^8, метод считается расходящимся.

На сетке 100x100 система плохо обусловлена, поэтому Зейделю и SOR с неудачным ω может не хватить `MAX_ITER` итераций. Это не ошибка, а результат эксперимента (статус `max_iter`); при желании увеличьте `MAX_ITER`.

In [5]:
```python
def stationary_solve(A, b, apply_Minv, tol=TOL, max_iter=MAX_ITER, diverge=1e8):
    """x_{k+1} = x_k + M^{-1} (b - A x_k). Возвращает x, историю невязки, статус."""
    x = np.zeros(A.shape[0])
    nb = np.linalg.norm(b)
    r = b - A @ x
    hist = [np.linalg.norm(r) / nb]
    status = "max_iter"
    for _ in range(max_iter):
        x = x + apply_Minv(r)
        r = b - A @ x
        rel = np.linalg.norm(r) / nb
        hist.append(rel)
        if not np.isfinite(rel) or rel > diverge:
```

```
            status = "diverged"
            break
        if rel < tol:
            status = "converged"
            break
    return x, np.array(hist), status


def jacobi(A, b, **kw):
    d = A.diagonal()
    return stationary_solve(A, b, lambda r: r / d, **kw)


def _triangular_solver(A, omega):
    D = sp.diags(A.diagonal())
    L = sp.tril(A, k=-1)
    M = (D / omega + L).tocsc()
    return spla.splu(M, permc_spec="NATURAL", diag_pivot_thresh=0.0).solve


def seidel(A, b, **kw):
    return stationary_solve(A, b, _triangular_solver(A, 1.0), **kw)


def sor(A, b, omega, **kw):
    return stationary_solve(A, b, _triangular_solver(A, omega), **kw)


def steepest_descent(A, b, tol=TOL, max_iter=MAX_ITER):
    x = np.zeros(A.shape[0]); r = b - A @ x; nb = np.linalg.norm(b)
    hist = [np.linalg.norm(r) / nb]; status = "max_iter"
    for _ in range(max_iter):
        Ar = A @ r
        a = (r @ r) / (r @ Ar)
        x = x + a * r
        r = r - a * Ar
        hist.append(np.linalg.norm(r) / nb)
        if hist[-1] < tol:
            status = "converged"; break
    return x, np.array(hist), status


def conjugate_gradient(A, b, tol=TOL, max_iter=MAX_ITER):
    x = np.zeros(A.shape[0]); r = b - A @ x; s = r.copy(); nb = np.linalg.norm(b)
    rr = r @ r
    hist = [np.sqrt(rr) / nb]; status = "max_iter"
    for _ in range(max_iter):
        As = A @ s
        t = rr / (s @ As)
        x = x + t * s
        r = r - t * As
        rr_new = r @ r
        hist.append(np.sqrt(rr_new) / nb)
        if hist[-1] < tol:
            status = "converged"; break
        s = r + (rr_new / rr) * s
```

```
        rr = rr_new
    return x, np.array(hist), status


def dominance_report(A):
    """Слабое/строгое диагональное преобладание по строкам."""
    absA = abs(A)
    d = np.abs(A.diagonal())
    off = np.asarray(absA.sum(axis=1)).ravel() - d
    return bool(np.all(d >= off)), bool(np.all(d > off))
```

2.1 Проверка реализаций на матрице со строгим диагональным преобладанием

Это тест из вашего исходного кода: для такой матрицы Якоби и Зейдель сходятся гарантированно. Сравниваем с прямым решением `numpy.linalg.solve`.

In [6]:
```
def random_dominant(n, rng):
    A = rng.integers(-50, 100, (n, n)).astype(float)
    np.fill_diagonal(A, 0.0)
    s = np.abs(A).sum(axis=1)
    np.fill_diagonal(A, s * rng.uniform(1.2, 2.0, n) + 1.0)  # строгое преобл.
    return sp.csr_matrix(A)


A_d = random_dominant(200, rng)
b_d = rng.integers(1, 100, 200).astype(float)
x_ref = np.linalg.solve(A_d.toarray(), b_d)
print("слабое/строгое преобладание:", dominance_report(A_d))

for name, res in [("Якоби", jacobi(A_d, b_d, tol=1e-12)),
                  ("Зейдель", seidel(A_d, b_d, tol=1e-12)),
                  ("SOR w=1.1", sor(A_d, b_d, 1.1, tol=1e-12))]:
    x, hist, status = res
    print(f"{name:10s} итераций: {len(hist)-1:4d}  статус: {status:9s}  "
          f"макс. отклонение от точного: {np.max(np.abs(x - x_ref)):.2e}")
```

```
слабое/строгое преобладание: (True, True)
Якоби      итераций:   28  статус: converged  макс. отклонение от точного: 3.94e-15
Зейдель    итераций:   12  статус: converged  макс. отклонение от точного: 9.09e-16
SOR w=1.1  итераций:   16  статус: converged  макс. отклонение от точного: 5.80e-15
```

2.2 Матрица из PyAMG (линейная упругость, МКЭ)

Берём `pyamg.gallery.linear_elasticity` на сетке `GRID x GRID` (не меньше 100x100, как в задании) и латунь ($E = 110$ ГПа, $\nu = 0.34$).

Две особенности, ради которых в функции есть проверки:

1. Из-за $E \sim 10^{11}$ элементы матрицы огромны. Делим матрицу на среднее диагонали: это масштабирует систему и не меняет сходимость методов, так как невязка относительная.

2. Матрица жёсткости без закреплений вырождена (твёрдотельные смещения и поворот дают нулевые собственные значения). Функция оценивает наименьшее собственное значение и, если оно близко к нулю, добавляет малый сдвиг `ALPHA * I`. Для матрицы PyAMG вырожденности нет (граничные условия уже учтены), поэтому сдвиг обычно не добавляется. Что именно сделано, печатается.

In [7]:
```
def elasticity_matrix(grid, nu=NU, E=E_MOD, alpha=ALPHA, verbose=True):
    import pyamg
    A, _ = pyamg.gallery.linear_elasticity((grid, grid), spacing=(1.0, 1.0),
                                           E=E, nu=nu, format="csr")
    A = sp.csr_matrix(A)
    asym = abs(A - A.T).max() / abs(A).max()
    A = ((A + A.T) / 2).tocsr()
    A = A / A.diagonal().mean()                        # масштабирование
    lam_min = spla.eigsh(A, k=1, sigma=-1e-4, which="LM",
                         return_eigenvectors=False)[0]
    shifted = lam_min < 1e-8
    if shifted:
        A = (A + alpha * sp.identity(A.shape[0])).tocsr()
    if verbose:
        print(f"размер: {A.shape[0]} x {A.shape[1]}, ненулевых: {A.nnz}")
        print(f"несимметричность до симметризации: {asym:.2e}")
        note = (f"-> добавлен сдвиг {alpha}*I" if shifted
                else "-> матрица невырождена, сдвиг не нужен")
        print(f"наименьшее собственное значение до сдвига: {lam_min:.3e}", note)
        print("слабое/строгое диагональное преобладание:", dominance_report(A))
    return A


A = elasticity_matrix(GRID)
b = rng.integers(1, 100, A.shape[0]).astype(float)
```

```
размер: 20000 x 20000, ненулевых: 256016
несимметричность до симметризации: 0.00e+00
наименьшее собственное значение до сдвига: 6.590e-04 -> матрица невырождена, сдвиг н
е нужен
слабое/строгое диагональное преобладание: (False, False)
```

In [8]:
```
def timed(label, func, *args):
    t0 = time.perf_counter()
    res = func(*args)
    print(f"{label} готов", round(time.perf_counter() - t0, 1), "c")
    return res


runs = {}
runs["Якоби"] = timed("Якоби", jacobi, A, b)
runs["Зейдель"] = timed("Зейдель", seidel, A, b)
for W in (1.5, 1.97):
    runs[f"Верхняя релаксация (w={W})"] = timed(f"SOR w={W}", sor, A, b, W)
runs["Наискорейший спуск"] = timed("СП", steepest_descent, A, b)
runs["Сопряжённые градиенты"] = timed("СГ", conjugate_gradient, A, b)

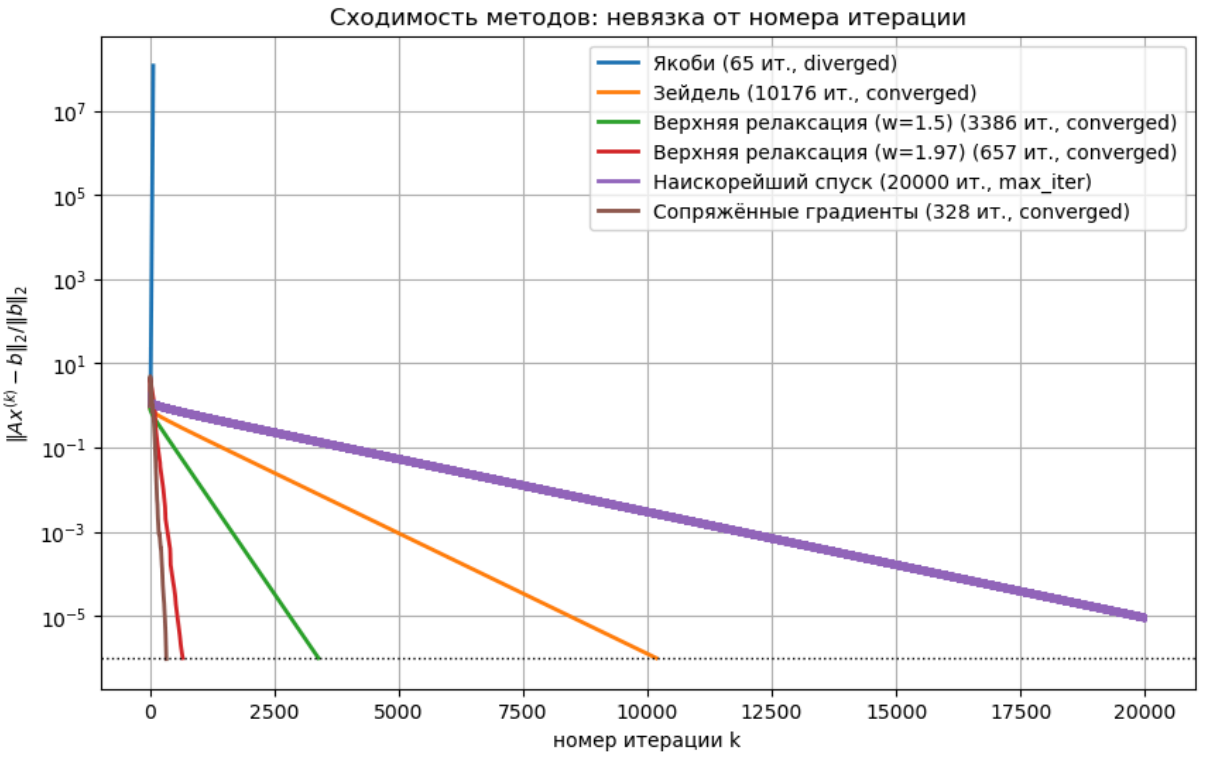
print()
```

```
print(f"{'метод':32s}{'итераций':>10s}{'статус':>12s}{'итоговая невязка':>20s}")
for name, (x, hist, status) in runs.items():
    print(f"{name:32s}{len(hist)-1:10d}{status:>12s}{hist[-1]:20.3e}")
```

```
Якоби готов 0.1 с
Зейдель готов 24.5 с
SOR w=1.5 готов 8.4 с
SOR w=1.97 готов 3.7 с
СП готов 23.8 с
СГ готов 0.3 с

метод                                итераций      статус    итоговая невязка
Якоби                                      65    diverged           1.169e+08
Зейдель                                 10176   converged           9.999e-07
Верхняя релаксация (w=1.5)               3386   converged           9.997e-07
Верхняя релаксация (w=1.97)               657   converged           9.986e-07
Наискорейший спуск                      20000    max_iter           8.021e-06
Сопряжённые градиенты                     328   converged           9.465e-07
```

In [9]:
```
plt.figure()
for name, (x, hist, status) in runs.items():
    plt.semilogy(hist, label=f"{name} ({len(hist)-1} ит., {status})", linewidth=2)
plt.axhline(TOL, color="k", linestyle=":", linewidth=1)
plt.title("Сходимость методов: невязка от номера итерации")
plt.xlabel("номер итерации k")
plt.ylabel(r"$\|Ax^{(k)} - b\|_2 / \|b\|_2$")
plt.legend()
plt.show()
```



3. Десерт: метод верхней релаксации и параметр ω

Для скорости считаем на меньших сетках (число итераций сильно растёт с размером). Если метод не сошёлся за лимит итераций, на графике это пропуск.

In [10]:
```
SCAN_MAX_ITER = 8000

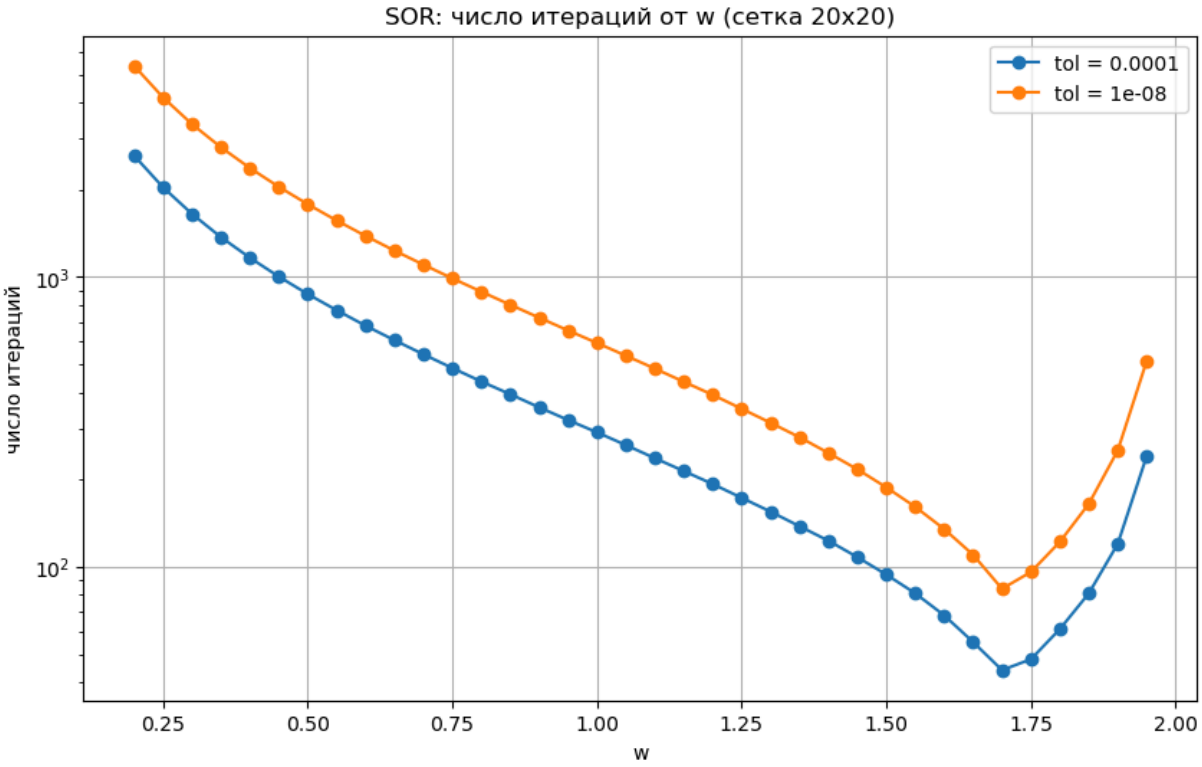
def sor_iterations(A, b, omega, tol, max_iter=SCAN_MAX_ITER):
    _, hist, status = sor(A, b, omega, tol=tol, max_iter=max_iter)
    return len(hist) - 1 if status == "converged" else np.nan


def find_omega_opt(A, b, tol, max_iter=SCAN_MAX_ITER):
    """Перебор w с шагом 0.1, затем уточнение с шагом 0.01 около лучшего."""
    coarse = np.round(np.arange(0.2, 1.99, 0.1), 2)
    its = np.array([sor_iterations(A, b, w, tol, max_iter) for w in coarse])
    if np.all(np.isnan(its)):
        return np.nan, np.nan
    w0 = coarse[np.nanargmin(its)]
    lo, hi = max(0.05, w0 - 0.1), min(1.99, w0 + 0.1) + 1e-9
    fine = np.round(np.arange(lo, hi, 0.01), 2)
    its_f = np.array([sor_iterations(A, b, w, tol, max_iter) for w in fine])
    i = np.nanargmin(its_f)
    return fine[i], its_f[i]


GRID_SCAN = 20
A_s = elasticity_matrix(GRID_SCAN)
b_s = rng.integers(1, 100, A_s.shape[0]).astype(float)

omegas = np.round(np.arange(0.2, 1.99, 0.05), 2)
plt.figure()
for tol in (1e-4, 1e-8):
    its = [sor_iterations(A_s, b_s, w, tol) for w in omegas]
    plt.plot(omegas, its, "o-", label=f"tol = {tol:g}")
    best = omegas[np.nanargmin(its)]
    print(f"tol={tol:g}: лучшее w (шаг 0.05) -> {best}, "
          f"итераций {np.nanmin(its):.0f}")
plt.yscale("log")
plt.xlabel("w"); plt.ylabel("число итераций")
plt.title(f"SOR: число итераций от w (сетка {GRID_SCAN}x{GRID_SCAN})")
plt.legend(); plt.show()
```

```
размер: 800 x 800, ненулевых: 9616
несимметричность до симметризации: 0.00e+00
наименьшее собственное значение до сдвига: 1.518e-02 -> матрица невырождена, сдвиг н
е нужен
слабое/строгое диагональное преобладание: (False, False)
tol=0.0001: лучшее w (шаг 0.05) -> 1.7, итераций 44
tol=1e-08: лучшее w (шаг 0.05) -> 1.7, итераций 84
```



3.1 Зависит ли ω_{opt} от точности и размера?

Для каждой пары (сетка, точность) находим ω_{opt} перебором. Дополнительно сравниваем с формулой $\omega = 2/(1+\sqrt{1-\rho_J^2})$. Она строго верна только для согласованно упорядоченных матриц и при $\rho_J < 1$; если $\rho_J \geq 1$ (Якоби расходится), в таблице стоит `nan`, и формула неприменима.

In [11]:
```python
def jacobi_spectral_radius(A):
    d = A.diagonal()
    Dh = sp.diags(1 / np.sqrt(d))
    S = Dh @ A @ Dh                          # D^-1/2 A D^-1/2
    lmin = spla.eigsh(S, k=1, which="SA", return_eigenvectors=False)[0]
    lmax = spla.eigsh(S, k=1, which="LA", return_eigenvectors=False)[0]
    return max(abs(1 - lmin), abs(1 - lmax))

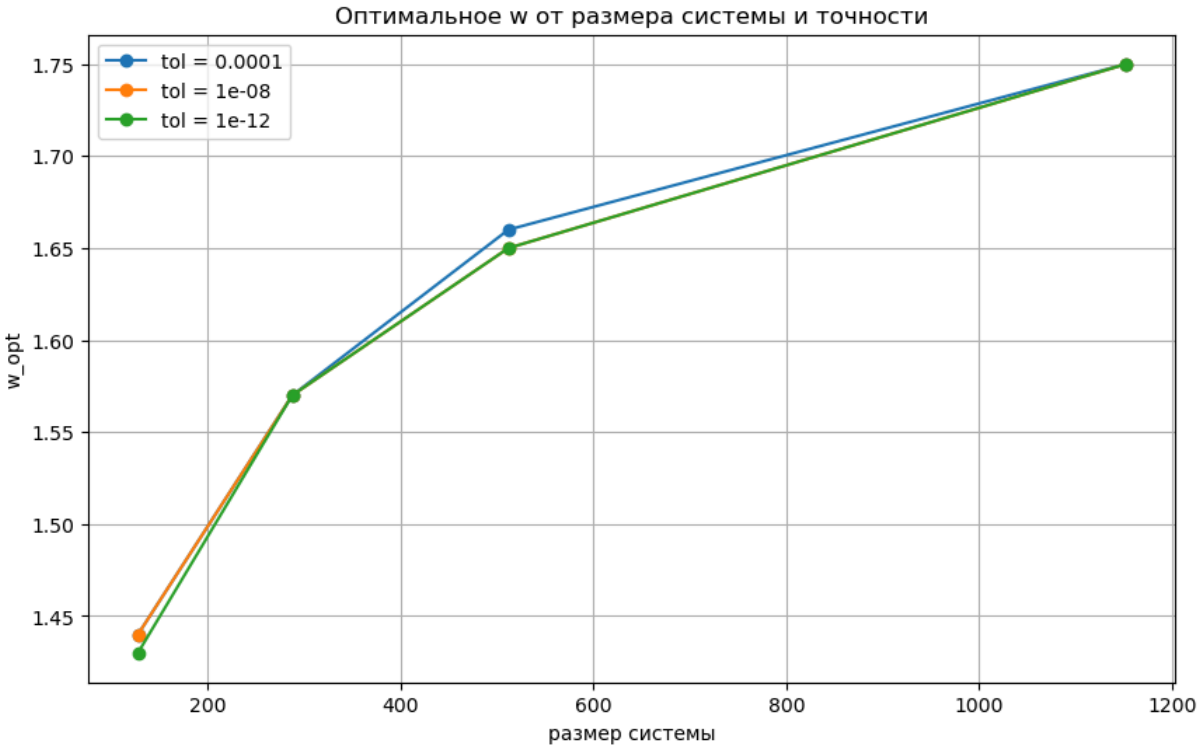

rows = []
for g in (8, 12, 16, 24):
    Ag = elasticity_matrix(g, verbose=False)
    bg = np.random.default_rng(g).integers(1, 100, Ag.shape[0]).astype(float)
    rho = jacobi_spectral_radius(Ag)
    w_theory = 2 / (1 + np.sqrt(1 - rho ** 2)) if rho < 1 else np.nan
    for tol in (1e-4, 1e-8, 1e-12):
        w_opt, it_opt = find_omega_opt(Ag, bg, tol)
        rows.append((g, Ag.shape[0], tol, w_opt, it_opt, rho, w_theory))

print(f"{'сетка':>6s}{'размер':>8s}{'tol':>9s}{'w_opt':>8s}{'итер.':>8s}"
      f"{'rho_J':>8s}{'w по формуле':>14s}")
```

```
for g, n_, tol, w_opt, it_opt, rho, wt in rows:
    print(f"{g:6d}{n_:8d}{tol:9.0e}{w_opt:8.2f}{it_opt:8.0f}{rho:8.4f}{wt:14.3f}")
```

```
сетка  размер      tol    w_opt    итер.    rho_J  w по формуле
    8     128    1e-04     1.44       18  1.3021           nan
    8     128    1e-08     1.44       35  1.3021           nan
    8     128    1e-12     1.43       53  1.3021           nan
   12     288    1e-04     1.57       26  1.3597           nan
   12     288    1e-08     1.57       50  1.3597           nan
   12     288    1e-12     1.57       75  1.3597           nan
   16     512    1e-04     1.66       34  1.3823           nan
   16     512    1e-08     1.65       66  1.3823           nan
   16     512    1e-12     1.65      100  1.3823           nan
   24    1152    1e-04     1.75       51  1.3996           nan
   24    1152    1e-08     1.75       97  1.3996           nan
   24    1152    1e-12     1.75      147  1.3996           nan
```

In [12]:
```
plt.figure()
for tol in (1e-4, 1e-8, 1e-12):
    sel = [r for r in rows if r[2] == tol]
    plt.plot([r[1] for r in sel], [r[3] for r in sel], "o-", label=f"tol = {tol:g}"
plt.xlabel("размер системы"); plt.ylabel("w_opt")
plt.title("Оптимальное w от размера системы и точности")
plt.legend(); plt.show()
```



3.2 Влияют ли другие параметры матрицы? Пример: коэффициент Пуассона ν

Модуль Юнга E только масштабирует матрицу и на ω_{opt} не влияет (мы всё равно нормируем). Коэффициент ν меняет связь между компонентами смещений, то есть структуру матрицы.

In [13]:
```
print(f"{'nu':>6s}{'w_opt':>8s}{'итераций':>10s}")
for nu_ in (0.1, 0.25, 0.34, 0.45):
    An = elasticity_matrix(GRID_SCAN, nu=nu_, verbose=False)
    bn = np.random.default_rng(1).integers(1, 100, An.shape[0]).astype(float)
    w_opt, it_opt = find_omega_opt(An, bn, 1e-8)
    print(f"{nu_:6.2f}{w_opt:8.2f}{it_opt:10.0f}")
```

```
   nu   w_opt  итераций
 0.10    1.71        73
 0.25    1.71        76
 0.34    1.71        81
 0.45    1.72       146
```

Выводы

Результаты получены для матрицы `pyamg.gallery.linear_elasticity` (латунь, сетка 100×100, размер системы 20000×20000, `pyamg` 5.3.0); для исследования ω использованы меньшие сетки.

Разминка. Метод прогонки восстановил точное решение $\bar{x} = [0, 1, \ldots, n]$ с максимальной относительной погрешностью порядка 10^{-15} по всем N, то есть на уровне ошибок округления; `scipy.sparse.linalg.spsolve` даёт тот же порядок. Наклон графика времени в log-log близок к 1 (в разных запусках от 0,9 до 1,1), что согласуется с линейной сложностью O(N). Небольшие отклонения вызваны шумом измерений и влиянием памяти при больших N, а при малых N время порядка микросекунд сравнимо с погрешностью таймера, поэтому точки там «шумят».

Основное блюдо. Реализации Якоби, Зейделя и SOR проверены на матрице со строгим диагональным преобладанием: все методы сошлись к решению `numpy.linalg.solve` (отклонение около 10^{-15}), поэтому поведение методов на матрице упругости определяется свойствами матрицы, а не ошибкой в коде.

Матрица упругости симметрична и положительно определена (наименьшее собственное значение $6{,}6 \cdot 10^{-4}$, сдвиг диагонали не потребовался), но диагонального преобладания не имеет. Результаты при tol = 10^{-6}:

метод	итераций	статус
Якоби	65	расходится
Зейдель	10176	сошёлся
SOR, ω = 1,5	3386	сошёлся
SOR, ω = 1,97	657	сошёлся
наискорейший спуск	20000	не достиг tol (невязка $8 \cdot 10^{-6}$)
сопряжённые градиенты	328	сошёлся

Якоби расходится: спектральный радиус его матрицы перехода ρ_J = 1,30–1,40 > 1 (таблица в разделе 3.1), а диагонального преобладания нет. Зейдель и SOR сходятся, так как матрица симметрична и положительно определена (для SOR достаточно 0 < ω < 2). Выбор ω сильно ускоряет метод: SOR с ω = 1,97 примерно в 15 раз быстрее Зейделя. Сопряжённые градиенты быстрее всех: число их итераций растёт примерно как $\sqrt{\kappa}$, а у наискорейшего спуска и метода Зейделя как κ (κ — число обусловленности), поэтому наискорейший спуск за лимит итераций не дошёл до заданной точности.

Десерт. Число итераций SOR как функция ω имеет выраженный минимум: слева от него оно убывает почти линейно в логарифмическом масштабе, справа резко растёт. На сетке 20×20 минимум около ω ≈ 1,7 (44 итерации при tol = 10^{-4} и 84 при tol = 10^{-8}).

С ростом размера системы ω_{opt} растёт: 1,44 (размер 128), 1,57 (288), 1,65 (512), 1,75 (1152), то есть стремится к 2 при h → 0. От требуемой точности ω_{opt} почти не зависит (различия до 0,01, в пределах шага перебора), а число итераций растёт примерно пропорционально числу верных знаков (для размера 128: 18, 35 и 53 итерации при tol = 10^{-4}, 10^{-8} и 10^{-12}).

Формула $\omega = 2/(1+\sqrt{1-\rho^2})$ здесь неприменима: ρ_J > 1, а матрица не является согласованно упорядоченной, поэтому ω_{opt} определялся только экспериментально.

Коэффициент Пуассона ν слабо влияет на ω_{opt} (1,71–1,72), но заметно на число итераций: от 73 при ν = 0,1 до 146 при ν = 0,45; при ν → 0,5 (несжимаемый материал) матрица обусловлена хуже. Модуль Юнга E лишь масштабирует матрицу и на результат не влияет.